

УЧЕБНИК ДЛЯ НАЧИНАЮЩИХ

ОСНОВЫ программирования на Python

С нуля до своих программ и игр. Для тех, кто ещё ни разу не писал код.

Часть 0 · Как думает программа

```
print("Привет! Я начинаю учить Python.")
```

9 класс · субботний курс · Эсхата Академия

Весь путь за год

0 Как думает программа

Урок 0.1 — Первые команды. print и кавычки

эта часть

Урок 0.2 — Переменные: ящики для данных

эта часть

👉 Вы держите Часть 0. Это два коротких урока, которые объясняют самое главное — как вообще работает программа. После них всё остальное пойдёт легко.

1 Первые программы

Ввод, числа, условия, циклы, строки

скоро

Проекты: «Угадай число», «Виселица»

скоро

2 Структуры и функции

Списки, функции, словари

скоро

Проект: «Крестики-нолики»

скоро

3 Графика и большой проект

turtle, файлы

скоро

Проект: «Змейка»

скоро

4 Данные и свой проект

Анализ текста, обработка ошибок, финальный проект

скоро

Первые команды: программа делает то, что сказано

ЧЕМУ НАУЧИМСЯ

- Понять, что программа — это команды, которые компьютер выполняет по порядку
- Написать первую команду **print** и увидеть результат
- Разобраться, зачем нужны кавычки



Что такое программа

Программа — это **список команд для компьютера**. Компьютер читает их сверху вниз, одну за другой, и выполняет ровно то, что написано. Он не догадывается и не додумывает — делает буквально.



На пальцах. Представь, что объясняешь другу, как заварить чай: налей воду, вскипяти, положи заварку, подожди. Это команды по порядку. Если перепутать шаги — чай не получится. Программа — то же самое, только компьютер ещё строже: он не поймёт намёков, ему нужна точность.



Первая команда — print

Команда **print** означает «покажи на экране». То, что стоит в скобках и кавычках, компьютер выведет как есть:

```
privet.py

print("Привет!")
print("Меня зовут Python.")
print("Я выполняю команды по порядку.")
```

НА ЭКРАНЕ ПОЯВИТСЯ

Привет!

Меня зовут Python.

Я выполняю команды по порядку.

Три команды — три строки на экране, ровно в том порядке, как написаны. Поменяешь строки местами — изменится и порядок вывода. Вот что значит «программа = команды по порядку».



ЗАДАНИЕ В ПАРЕ

Расскажи о себе

Напишите программу из трёх команд `print` : пусть она выведет твоё имя, твой город и твой возраст — каждое на своей строке.

Водитель печатает, штурман диктует. Каждую строку сразу запускайте клавишей F5.



Главное открытие: кавычки

А теперь фокус. Наберите две очень похожие строки и запустите:

 kavychki.py

```
print("2 + 2")
print(2 + 2)
```

НА ЭКРАНЕ ПОЯВИТСЯ

```
2 + 2
4
```

ЗАПОМНИ ЭТО ПРАВИЛО

Кавычки говорят: **"покажи ровно эти значки"**. Без кавычек компьютер думает, что это задачка, и **решает** её. Кавычки = «как есть», без кавычек = «посчитай».



Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>print(Привет)</code>	<code>print("Привет")</code>
<code>print('Привет')</code>	<code>print("Привет")</code>
<code>prin("Привет")</code>	<code>print("Привет")</code>

Компьютер не догадается, что ты имел в виду. Ошибка — это не провал, а подсказка: прочитай, что он пишет, и поправь.



ДОМАШНЕЕ ЗАДАНИЕ

1. Программа выводит твоё имя и три факта о тебе (три команды `print`).
2. Программа выводит расписание твоих уроков на завтра.

★ **Со звёздочкой:** нарисуй рамку из символов `*` и напиши внутри «Я учу Python».

Переменные: ящики для данных

ЧЕМУ НАУЧИМСЯ

- Понять, что такое переменная и зачем она нужна
- Класть значение в «ящик» и доставать его по имени
- Считать с помощью переменных



Зачем нужны переменные

На прошлом уроке мы писали всё прямо в `print`. Но что делать, если одно значение нужно использовать много раз? Каждый раз печатать заново? Нет — его можно **сохранить**.



На пальцах. Переменная — это **ящик с наклейкой**. На наклейке — имя, внутри — значение. Ты кладёшь что-то один раз, а достаёшь столько раз, сколько нужно. Захотел поменять содержимое — положил новое, старое пропало.



Кладём в ящик

Знак `=` здесь означает не «равно», как в математике, а **«положи в ящик»**. Слева — имя ящика, справа — что кладём:

yaschiki.py

```
age = 15
print(age)

name = "Алишер"
print(name)
```

НА ЭКРАНЕ ПОЯВИТСЯ

15
Алишер

Мы написали имя ящика — и компьютер достал из него значение. Читается так: «ящик `age` получает значение 15».



Зачем это удобно

Один раз положил — используешь сколько угодно раз:

privet_imya.py

```
name = "Алишер"
print("Привет,")
print(name)
print("тебе лет:")
print(age)
```

И самое полезное — с числами в ящиках можно **считать**. Вот расчёт покупки:

```
рокупка.py

price = 25      # сомони за штуку
count = 4       # сколько штук
print(price * count)
```

НА ЭКРАНЕ ПОЯВИТСЯ
100

ЗАПОМНИ ЭТО ПРАВИЛО

Знак `=` — это «положи в ящик», а не «равно». Пишешь имя ящика — компьютер достаёт значение. Новое значение стирает старое.

👉 ЗАДАНИЕ В ПАРЕ

Средний балл

Положите пять своих оценок в ящики `g1 ... g5`, а потом выведите средний балл: сумму разделите на 5. Подсказка: `print((g1 + g2 + g3 + g4 + g5) / 5)`

Через 10 минут поменяйтесь ролями — теперь печатает штурман.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
print(Name)
```

```
age = "15"
print(age + 1)
```

```
print(age)
age = 15
```

А ТАК ПРАВИЛЬНО

```
print(name)
```

```
age = 15
print(age + 1)
```

```
age = 15
print(age)
```

Python различает большие и маленькие буквы: `name` и `Name` — разные ящики. И сначала клади в ящик, потом доставай — не наоборот.



ДОМАШНЕЕ ЗАДАНИЕ

1. Создай ящики для профиля: ник, город, любимый предмет. Выведи их красиво.
2. Есть цена товара и количество — посчитай общую стоимость в сомони.

★ **Со звездочкой:** положи в ящики цены трёх покупок и выведи, сколько всего денег нужно.